Assignment 2

Blockchain Technologies

Overview

Task: AI Model Marketplace dApp

Objective:

Create a decentralized application that allows users to list, purchase, and rate AI models. The smart contract should manage the core functionality, while the frontend should provide an intuitive interface for users to interact with the contract.

Smart Contract Functions

`listModel(string memory name, string memory description, uint256 price)` - Allows users to add a new AI model to the marketplace with a name, description, and price.

`purchaseModel(uint256 modelId)` - Enables users to buy a specific AI model by its ID, transferring the payment to the model's creator.

`rateModel(uint256 modelId, uint8 rating)` - Lets users rate a purchased AI model, contributing to its overall rating score.

`withdrawFunds()` - Allows the contract owner to withdraw accumulated funds from model sales.

`getModelDetails(uint256 modelId)` - Retrieves and returns the details of a specific AI model, including its name, description, price, creator, and average rating.

Frontend

- The frontend should be a simple web application with the following components:
- A form to list a new AI model (connects to `listModel` function)
- A list or grid of available models
- A button to purchase a model (connects to `purchaseModel` function)
- A form to rate a purchased model (connects to `rateModel` function)
- A button to view model details (connects to `getModelDetails` function)
- A button for model creators to withdraw their funds (connects to `withdrawFunds` function)

Project Repository structure

- Source code
- README.md
- - Title
  - Usage
    - Put here demo screenshots (png or gif)
  - Examples
- LICENSE
  - For example: https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/LICENSE

Create repository in github, push it into the github and submit the link to the repository into the moodle. Assignment will not be accepted if Repository and README are not submitted and structured properly!!!

References

- 1. https://docs.web3js.org/ - Documentation to web3 js
- 2. https://docs.web3js.org/guides/smart_contracts/smart_contracts_guide - Deploying and Interacting with Smart Contracts
- 3. https://trufflesuite.com/ganache/ - Ganache blockchain
- 5. https://medium.com/@kacharlabhargav21/using-ganache-with-remix-and-metamask-446fe5748ccf - How to configure Remix with Ganache and Metamask

Grading Policy

Category	Points
Smart Contract Development	**30**
Deployment	**30**
Deployment to Testnet	30
Deployment to Ganache	10
Frontend Integration with Web3.js	**30**
Readme	**10**